

Тема № 11. Наследование классов

11.1. Общие сведения о наследовании

Основная идея: на базе существующего класса (класс-родитель или базовый класс), создается производный класс (класс-наследник, дочерний класс), который включает в себя поля и методы базового класса (наследует их), и содержит дополнительные поля (обладает новыми свойствами) и методы.

11.2. Определение производного класса

Примеры:

```
class S: X, Y, Z { ... } ;
```

```
class B: public A {....};
```

```
class D: public X, protected B  
{....};
```

Перед именем базового класса может указываться статус доступа наследования или тип наследования доступа (одно из ключевых слов `public`, `protected`, `private`). Статус доступа наследования определяет статус доступа наследуемых полей и функций из базового класса внутри производного класса.

Производный класс может определяться с ключевым словом `struct` или `class` (с ключевым словом `union` производный класс не определяется). Если производный класс определен с ключевым словом `class`, то по умолчанию статус доступа наследования `private`.

Если производный класс определен с ключевым словом `struct`, то по умолчанию статус доступа наследования `public`.

11.3. Статусы доступа при наследовании классов

Тип наследования доступа	Доступность в базовом классе	Доступность компонент базового класса в производном
public	public	public
	protected	protected
	private	не доступны
protected	public	protected
	protected	protected
	private	не доступны
private	public	private
	protected	private
	private	не доступны

11.4. Особенности конструкторов при наследовании

Конструктор производного класса в первую очередь всегда должен вызывать конструктор базового класса. Если это действие не выполняется явно, то по умолчанию вызывается конструктор без параметров (если он есть, если его нет, будет ошибка). Если класс имеет несколько базовых, то конструкторы базовых классов должны вызываться в порядке перечисления этих классов в списке базовых.

```
#include <cstdlib>
#include <iostream>
using namespace std;

class A
{
public:
    A()
    {
        cout<<endl<<"A1";
    }
    A(int x)
    {
        cout<<endl<<"A2";
    }
};

class B: public A
{
public:
    B(): A(4) // Явный вызов конструктора с 1-м параметром, если нет явного вызова, то
              // вызывается конструктор без параметров, если его нет, будет ошибка
    {
        cout<<endl<<"B";
    }
};

class C
{
public:
    /*C()
    {
        cout<<endl<<"C1";
    }*/
    C(int x)
    {
        cout<<endl<<"C2";
    }
};

class D: public A, public C
{
public:
    D() : A(3), C(5) // Явный вызов конструкторов
    {
        cout<<endl<<"D";
    }
};
```

```

int main(int argc, char* argv[])
{
    B b1;
    D d1;
    system("pause");
    return 0;
}

```

11.5. Особенности деструкторов при наследовании

Деструктор производного класса всегда неявно по умолчанию после выполнения своего тела вызывает деструкторы базовых классов. Причем порядок разрушения объекта (вызовов деструкторов) обратен порядку создания (вызова конструкторов).

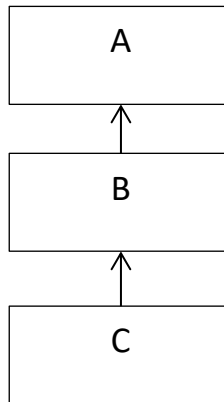
```

#include <iostream>
using namespace std;
class A
{
public:
    ~A()
    {
        cout<<endl<<"A";
    }
};
class B
{
public:
    ~B()
    {
        cout<<endl<<"B";
    }
};
class C: public A, public B
{
public:
    ~C()
    {
        cout<<endl<<"C";
    }
};
int main(int argc, char* argv[])
{
    {
        C c1;
    } // При выходе из блока объект c1 уничтожается - вызывается деструктор
    return 0;
}

```

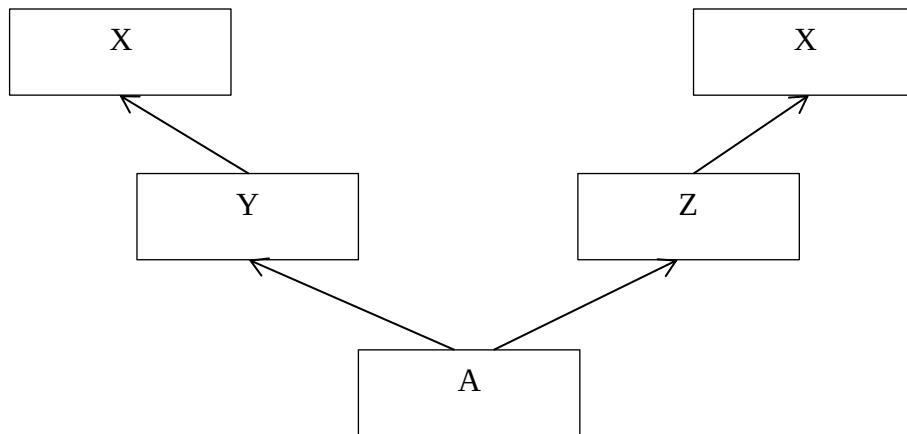
11.6. Множественное наследование и виртуальные базовые классы

Представление отношения наследования в виде диаграммы классов.



Множественное наследование – это когда класс имеет в качестве базовых более одного класса.

Пример диаграммы



Пример заголовков классов

```
class X {  
    public: int k; void f() { }  
};  
class Y : public X { };  
class Z : public X { };  
class A : public Y, public Z { };  
int main()  
{  
    A a;  
    a.k = 10; // Здесь ошибка  
    return 0;  
}
```

При такой схеме наследования происходит дублирование полей и методов класса X в классе A. Обращение к полям и методам объекта класса A, наследуемым из класса X, является неоднозначным, как внутри методов объекта

класса А, так и за пределами класса через имя объекта. Поля и методы из класса Х наследуются классами Y и Z, а в объекте класса А существуют в 2-х экземплярах, наследуются из класса Y и Z. Компилятор при попытке обращения к таким полям и методам для объекта класс А выдает ошибку.

Для устранения возможность дублирования полей и методов, когда они наследуются из 1-го класса при множественном наследовании, введено понятие виртуального класса, в предыдущем примере класс Х должен быть объявлен как виртуальный, для этого используется ключевое слово `virtual`.

Виртуальный класс — класс, который при множественном наследовании дополнительно включается в классы-потомки ссылкой на объект этого класса во избежания дублирования. (Для обычного наследования в Си++ в оперативной памяти в объекте производного класса вначале идут поля базового класса, а затем поля производного, при виртуальном наследовании дополнительно появляется ссылка в производном классе).

Термин «виртуальный» собственно относится не к базовому классу, а к способу наследования от него.

```
class X {
    public: int k; void f() { }
};
class Y : public virtual X { };
class Z : public virtual X { };
class A : public Y, public Z { };

int main()
{
    A a;
    a.k = 10; // Ошибки нет
    return 0;
}
```

Следует отметить, что вызов конструктора для объекта базового класса Х при создании объекта класса А, осуществляется в конструкторе объекта класса А (явно или по умолчанию вызывается конструктор без параметров), он вызывается первым до вызова конструкторов Y и Z. При вызове конструкторов объектов классов Y и Z внутри конструктора объекта класса А в них конструктор объекта Х не вызывается, т.к. уже был вызван до этого. Пример явного вызова конструкторов:

```
#include <iostream>
using namespace std;
class X {
    public: int k; void f() { }
        X(int k) : k(k)
        {
            cout << "X(int k)" << endl;
        }
};
class Y : public virtual X {
public:
```

```

    Y(int k) : X(k)
    {
        cout << "Y(int k)" << endl;
    }
};
class Z : public virtual X {
public:
    Z(int k) : X(k)
    {
        cout << "Z(int k)" << endl;
    }
};

class A : public Y, public Z {
public:
    A(int k) : X(k), Y(k), Z(k)
    {
        cout << "A(int k)" << endl;
    }
};

int main()
{
    A a(10);
    return 0;
}

```

Будет напечатано:

X(int k)

Y(int k)

Z(int k)

A(int k)

11.7. Переопределение функций. Виртуальные функции

Если в производном классе объявлена функция с именем, типом возвращаемого значения и количеством и типами параметров, такая же, как в базовом классе, то данная функция является переопределенной. (Не путать с перегрузкой функций). С помощью простых переопределенных функций реализуется механизм статического полиморфизма. (Полиморфизм – возможность функции в производном классе работать по-другому).

Суть статического связывания: когда указатель одного типа ссылается на объект другого типа при наследовании классов, то выбор переопределенного метода определяется типом указателя, а не типом объекта.

```

#include <iostream>
using namespace std;
// Пример статического связывания
class A
{
    int a;
public:
    void print()

```

```

        {
            cout<<endl<<"A";
        }
    };
class B: public A
{
public:
    void print() // Переопределенная функция
    {
        cout<<endl<<"B";
    }
};
int main(int argc, char* argv[])
{
    B b;
    A *pA;
    pA=&b; // Преобразование типов выполняется по умолчанию (обратное не верно)
    pA->print(); // Статический полиморфизм. Вызов функции определяется типом указателя,
    а не типом объекта
    cout<<endl<<"sizeof(A)="<<sizeof(A)<<" sizeof(B)="<<sizeof(B);
    return 0;
}

```

Суть динамического связывания: когда указатель одного типа ссылается на объект другого типа при наследовании классов, то выбор переопределенного метода определяется типом объекта, а не типом указателя, для этого переопределенный метод должен быть объявлен виртуальным в базовом классе.

Встретив у функции модификатор `virtual` компилятор создает для класса таблицу виртуальных функций, а в класс добавляется новое скрытое для программиста поле — указатель на эту таблицу.

Таблица виртуальных функций хранит в себе адреса всех виртуальных методов класса (по сути, это массив указателей), а также всех виртуальных методов базовых классов этого класса.

```

#include <iostream>
using namespace std;
// Пример динамического связывания
class A
{
    int a;
public:
    virtual void print() // Функция в базовом классе объявлена как виртуальная
    {
        cout<<endl<<"A";
    }
};
class B: public A
{
public:
    void print() // Переопределенная функция
    {
        cout<<endl<<"B";
    }
}

```

```

    }
};
int main(int argc, char* argv[])
{
    B b;
    A *pA;
    pA=&b; // Преобразование типов выполняется по умолчанию (обратное не верно)
    pA->print(); // Динамический полиморфизм. Вызов функции определяется типом объекта,
а не типом указателя
    cout<<endl<<"sizeof(A)="<<sizeof(A)<<" sizeof(B)="<<sizeof(B);
// По размеру объекта видно, что есть дополнительное поле- указатель на таблицу вирт. функций
    return 0;
}

```

Каждый объект имеет дополнительное поле указатель на данную таблицу. Часто внутри переопределенного метода требуется (или удобно) вызвать этот метод из базового класса, в следующем примере демонстрируется эта возможность.

```

#include <iostream>
#include <cstdlib>
using namespace std;
class point // Базовый класс - "Точка на плоскости"
{
    double x, y; // Координаты точки
public:
    point(double x, double y) // Конструктор для инициализации полей
    {
        this->x = x; this->y = y;
    }
    virtual void print() // Метод для печати полей (виртуальный)
    {
        cout << "\nx=" << x << " y=" << y; // Печатаем значения полей
    }
};
class point3d : public point // Производный класс - "Точка в пространстве"
{
    double z; // Новое поле - координата z
public:
    point3d(double x, double y, double z) : // Конструктор
        point(x, y) // Явный вызов конструктора базового класса
    {
        this->z = z;
    }
    void print() // Переопределенный метод print
    {
        point::print(); // Вызов в переопределенном методе метода базового класса
        cout << " z=" << z; // Допечатывает поле z
    }
};

int main(int argc, char* argv[])
{
    point p1(1, 2); // Создается объект с вызовом конструктора
    point3d p3(3, 4, 5); // Создается объект с вызовом конструктора
    point *pp; // Указатель типа базового класса
}

```

```

    pp = &p1; // Настраиваем на объект базового класса
    pp->print(); // Вызов метода через указатель
    pp = &p3; // Настраиваем указатель на объект производного класса (преобразование типа
допустимо)
    pp->print(); // Вызов метода через указатель, вызывается метод класса point3d
    // Если метод print в классе point был объявлен без virtual, то вызывался бы метод
print класс point
    system("pause");
    return 0;
}

```

Ключевое слово override

Для виртуальных методов, которые переопределяются в производных классах, рекомендуется в заголовке метода в производном классе использовать модификатор ***override*** (переопределенный). Это требование не является обязательным, но позволяет избежать некоторых ошибок. Например, в коде

```

class A
{
public:
    virtual void fun() const
    { }
};

class B : public A
{
public:
    void fun()
    { }
};

```

Метод *fun* в классе *B* не является переопределенным, так как у него, в отличие от метода в базовом классе, отсутствует модификатор *const*. Компилятор считает, что это отдельный новый метод класса *B*, также метод *fun* с модификатором *const* наследуется из класса *A*. Но если в коде использовать в классе *B* *override*

```

class A
{
public:
    virtual void fun() const
    { }
};

class B : public A
{
public:
    void fun() override // Здесь ошибка
    { }
};

```

То компилятор выдаст ошибку, так как метод *fun* в *B*, должен обязательно переопределять метод из *A*. Ошибку можно исправить так:

```

class A

```

```

{
public:
    virtual void fun() const
    {    }
};

class B : public A
{
public:
    void fun() const override
    {    }
};

```

Или в двух классах *A* и *B* метод *fun* должен быть объявлен без модификатора *const*.

Виртуальный деструктор

Если в программе указатели базового класса могут ссылаться на объекты производных классов и при создании объектов производных классов дополнительно выделяется динамически память или используются ресурсы, которые нужно освободить, необходимо использовать виртуальный деструктор (объявляется явно).

Если этого не сделать, то возможны утечки памяти.

```

A *pA=new B();
delete pA; /* Если деструктор не виртуальной, то вызывается деструктор класса
A */

```

11.8. . Чистые виртуальные функции и абстрактные классы

Абстрактный класс это класс, который имеет в своем составе хотя бы одну чистую виртуальную функцию. ЧВФ не имеет тела и ничего не делает.

Чистая виртуальная функция:

```

virtual <тип_возвр_значения> <имя_функции>(<список_форм_парам>)=0;
virtual void fpure()=0;

```

Нельзя создать объект абстрактного класса (тип указателя на абстрактный класс может быть, этот указатель может содержать адрес объекта производного класса, не являющегося абстрактным). Абстрактный класс нужен, чтобы на его основе создавать обычные производные классы, в которых ЧВФ переопределяются (заменяются) на обычные функции.